

 BLOCK 1 – DSA: Stack / Containers — Service Packaging

 Core Idea

- Bundle your app and all dependencies
 - Ensure same environment everywhere
 - Use container (like stack of layers)
-

 Concept Mapping

Docker Image = Stack of layers

- Base image
 - Python
 - Dependencies
 - Your code
-

 Example (Docker layering idea)

FROM python:3.10

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

 LeetCode Problem LeetCode #155 — Min Stack

<https://leetcode.com/problems/min-stack/>

 Why this problem?

Stack → layered system

Push → add layers

Pop → remove layers

Docker works similarly: layered build → efficient reuse

🔍 Think

If environment differs: local works ✅ or server fails ❌

How do you guarantee consistency? → containerization ✅

🤖 BLOCK 2 – AI Concept: Deployment & Serving

✅ Problem

Your system runs locally ❌

Only you can use it

✅ Solution

Deploy API → public URL ✅

✅ Architecture

Client → Internet → FastAPI → RAG → Response

✅ Key Concepts

- API hosting (Render / AWS / Railway)
 - Port binding (0.0.0.0)
 - Scalability basics
 - Stateless API (sessions handled separately)
-

✅ Insight

Model - Product

Deployment = turning code into real service 

BLOCK 3 – Build: Deploy FastAPI with Docker

Step 1 — Create requirements.txt

fastapi

uvicorn

sentence-transformers

chromadb

rank-bm25

Step 2 — Create Dockerfile

FROM python:3.10

WORKDIR /app

COPY . .

RUN pip install --no-cache-dir -r requirements.txt

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

Step 3 — Build Docker image

docker build -t rag-app .

Step 4 — Run container

docker run -p 8000:8000 rag-app

Step 5 — Deploy (example: Render / Railway)

1. Push code to GitHub

2. Connect repo to Render

3. Set start command: `uvicorn main:app --host 0.0.0.0 --port 8000`

4. Deploy

- ✓ Verify
 - ✓ API works via public URL
 - ✓ /chat endpoint responds
 - ✓ multiple users supported
 - ✓ no local dependency issues
-

BLOCK 4 – Reflect

1. What happens if 100 users hit API at same time?
 2. How will you scale (multiple containers)?
 3. Where will you store vector DB in production?
 4. Should model run on CPU or GPU server?
 5. What happens if container crashes?
-

✓ ✓ SUMMARY

✓ DSA Mapping

Stack → layered container system

Efficient reuse of layers

✓ AI System

Local → Production ✓

API → Public ✓

Usable by others ✓
